

НЭСТ ЭДҮКЭЙШН АЙ ТИ ДУНД СУРГУУЛЬ



2025-2026 оны хичээлийн жил

TETRIS

ТӨСЛИЙН ТАНИЛЦУУЛГА

Гүйцэтгэсэн сурагчид:

Багийн ахлагч: Б.Тамир

Багийн гишүүд: О.Бат-Оргил

О.Замилан

УЛААНБААТАР ХОТ

2026 ОН

Агуулга

ТҮЛХҮҮР ҮГС, ТОВЧИЛСОН ҮГИЙН ЖАГСААЛТ.....	2
ТӨСЛИЙН ТАНИЛЦУУЛГА.....	2
Санаагаа хэрхэн олсон бэ?.....	2
Хэрхэн тоглох вэ?.....	2
ТӨСӨЛ БҮТЭЭХ ҮЙЛ ЯВЦ.....	3
Схем /Ардуино, Эд ангиуд, нийт холболт/.....	3
ТӨСЛИЙН ТЕСТ.....	5
Хүчдэл power.....	5
Ажиглалт, Орчноос хамаарсан өөрчлөлт.....	5
АШИГЛАСАН МАТЕРИАЛ.....	5
ТӨСЛИЙН ҮР ДҮН.....	5
Дүгнэлт.....	5
Шүүмж.....	6

ТҮЛХҮҮР ҮГС, ТОВЧИЛСОН ҮГИЙН ЖАГСААЛТ

Ардуино уно

Ардуино (Arduino UNO) гэдэг нь электрон төсөл, робот бүтээхэд зориулагдсан, програмчилж болдог жижиг хэмжээний ухаалаг удирдлагын хавтан юм.

IoT

“Internet Of Things” буюу бодит биет зүйлсийг программтай холбосон төрөл зүйл юм.

Joystick

Жойстик (Joystick) нь суурь дээр эргэлддэг саваа хэлбэрийн оролтын төхөөрөмж бөгөөд дижитал объектын хөдөлгөөнийг хянах, тоглоом тоглох, нисэх онгоц, гар удирдлага, үйлдвэрлэлийн машин механизмыг удирдахад ашиглагддаг төхөөрөмж юм.

Led

LED (Light Emitting Diode - Гэрэл ялгаруулах диод) нь цахилгаан гүйдлийг гэрэл болгон хувиргадаг удаан эдэлгээтэй гэрэлтүүлгийн технологи юм.

ТӨСЛИЙН ТАНИЛЦУУЛГА

Санаагаа хэрхэн олсон бэ?

Бид санаагаа 2023 оны “Tetris” гэх киног үзээд, үе тэнгийн найзууд болон ах эгч, дүү нар маань бас энэхүү гайхамшигтай тоглоомыг тоглож үзэх хэрэгтэй гэж бодон гаргасан.

Хэрхэн тоглох вэ?

Tetris-ийг тоглохдоо дээрээс унах янз бүрийн хэлбэртэй блокуудыг зүүн, баруун тийш хөдөлгөн эргүүлэн байрлуулан, нэг мөрийг бүхэлд нь дүүргэснээр тэр мөр алга болж оноо нэмэгдэнэ. Үүнийг хожигдох хүртэл давтана.

Хэрхэн ашиглах

Бидний төслийг ашиглахын тулд ханан дээр хөрөөдөн бэхжүүлж код аль хэдийн суулгасан Ардуинод тог өгөн шууд тоглож эхэлнэ.

Зорилго

Бид хойч үеийн хүүхэд багачуудад Tetris тоглоомыг танилцуулж, түүний сонирхолтой дүрэм, хурд, логик сэтгэлгээ шаардсан онцлогоор дамжуулан

хүүхдүүдийн анхаарал төвлөрөл, асуудал шийдвэрлэх гэх мэт чадварыг хөгжүүлэхэд нь туслаад зогсохгүй цаашлаад илүү өргөн хүрээнд түгээн дэлгэрүүлэхийг зорин ажиллах болно.

Код

Функц

Манай код нь C++ хэлээр хийгдсэн ба 596 мөрнөөс тогтдог. Тухайн код нь 19 функцээс тогтдог. Үүнд:

1. `bool getShape(uint8_t piece, uint8_t rot, uint8_t row, uint8_t col)` - туслагч функц
2. `int getLEDIndex(int row, int col)` - туслагч функц
3. `void playTone(int frequency, int duration)`
4. `void playMove()`
5. `void playRotate()`
6. `void playLineClear()`
7. `void playGameOver()`
8. `void updateLCD()`
9. `void showStartScreen()`
10. `void showGameOver()`
11. `void spawnPiece()`
12. `void rotatePiece(bool clockwise)`
13. `void hardDrop()`
14. `void clearLines()`
15. `void render()`
16. `void handleJoystick()`
17. `void handleButtons()`
18. `void restartGame()`
19. `void setup()`

```

// I piece
{
    {{0,0,0,0},{1,1,1,1},{0,0,0,0},{0,0,0,0}},
    {{0,0,1,0},{0,0,1,0},{0,0,1,0},{0,0,1,0}},
    {{0,0,0,0},{0,0,0,0},{1,1,1,1},{0,0,0,0}},
    {{0,1,0,0},{0,1,0,0},{0,1,0,0},{0,1,0,0}}
},
// O piece
{
    {{0,1,1,0},{0,1,1,0},{0,0,0,0},{0,0,0,0}},
    {{0,1,1,0},{0,1,1,0},{0,0,0,0},{0,0,0,0}},
    {{0,1,1,0},{0,1,1,0},{0,0,0,0},{0,0,0,0}},
    {{0,1,1,0},{0,1,1,0},{0,0,0,0},{0,0,0,0}}
},
// T piece
{
    {{0,1,0,0},{1,1,1,0},{0,0,0,0},{0,0,0,0}},
    {{0,1,0,0},{0,1,1,0},{0,1,0,0},{0,0,0,0}},
    {{0,0,0,0},{1,1,1,0},{0,1,0,0},{0,0,0,0}},
    {{0,1,0,0},{1,1,0,0},{0,1,0,0},{0,0,0,0}}
},
// S piece
{
    {{0,1,1,0},{1,1,0,0},{0,0,0,0},{0,0,0,0}},
    {{0,1,0,0},{0,1,1,0},{0,0,1,0},{0,0,0,0}},
    {{0,0,0,0},{0,1,1,0},{1,1,0,0},{0,0,0,0}},
    {{1,0,0,0},{1,1,0,0},{0,1,0,0},{0,0,0,0}}
},
// Z piece
{
    {{1,1,0,0},{0,1,1,0},{0,0,0,0},{0,0,0,0}},
    {{0,0,1,0},{0,1,1,0},{0,1,0,0},{0,0,0,0}},
    {{0,0,0,0},{1,1,0,0},{0,1,1,0},{0,0,0,0}},
    {{0,1,0,0},{1,1,0,0},{1,0,0,0},{0,0,0,0}}
},
// J piece
{
    {{1,0,0,0},{1,1,1,0},{0,0,0,0},{0,0,0,0}},
    {{0,1,1,0},{0,1,0,0},{0,1,0,0},{0,0,0,0}},
    {{0,0,0,0},{1,1,1,0},{0,0,1,0},{0,0,0,0}},
    {{0,1,0,0},{0,1,0,0},{1,1,0,0},{0,0,0,0}}
},

```

```

// L piece
{
    {{0,0,1,0},{1,1,1,0},{0,0,0,0},{0,0,0,0}},
    {{0,1,0,0},{0,1,0,0},{0,1,1,0},{0,0,0,0}},
    {{0,0,0,0},{1,1,1,0},{1,0,0,0},{0,0,0,0}},
    {{1,1,0,0},{0,1,0,0},{0,1,0,0},{0,0,0,0}}
}
};

// Helper function to read shape from PROGMEM
bool getShape(uint8_t piece, uint8_t rot, uint8_t row, uint8_t col) {
    return pgm_read_byte(&shapes[piece][rot][row][col]);
}

// Piece Colors
const CRGB colors[7] = {
    CRGB(0, 255, 255),    // I - Cyan
    CRGB(255, 255, 0),    // O - Yellow
    CRGB(160, 0, 255),    // T - Purple
    CRGB(0, 255, 0),      // S - Green
    CRGB(255, 0, 0),      // Z - Red
    CRGB(0, 0, 255),      // J - Blue
    CRGB(255, 136, 0)     // L - Orange
};

```

```

// ===== GAME LOGIC =====

void spawnPiece() {
    if (nextType == 0) {
        currentType = random(7);
        nextType = random(7);
    } else {
        currentType = nextType;
        nextType = random(7);
    }

    currentRotation = 0;
    currentX = COLS / 2 - 2;
    currentY = 0;
}

bool checkCollision(int x, int y, uint8_t type, uint8_t rotation) {
    for (int row = 0; row < 4; row++) {
        for (int col = 0; col < 4; col++) {
            if (getShape(type, rotation, row, col)) {
                int newX = x + col;
                int newY = y + row;

                if (newX < 0 || newX >= COLS || newY >= ROWS) {
                    return true;
                }

                if (newY >= 0 && grid[newY][newX] != 0) {
                    return true;
                }
            }
        }
    }
    return false;
}

bool movePiece(int dx, int dy) {
    int newX = currentX + dx;
    int newY = currentY + dy;

    if (!checkCollision(newX, newY, currentType, currentRotation)) {
        currentX = newX;
        currentY = newY;
        if (dx != 0 || dy != 0) playMove();
        return true;
    }
}

```

```

    return false;
}

void rotatePiece(bool clockwise) {
    uint8_t newRotation = clockwise ? (currentRotation + 1) % 4 : (currentRotation + 3) % 4;

    if (!checkCollision(currentX, currentY, currentType, newRotation)) {
        currentRotation = newRotation;
        playRotate();
    } else {
        // Wall kick attempts
        int kicks[][2] = {{1, 0}, {-1, 0}, {2, 0}, {-2, 0}, {0, -1}};
        for (int i = 0; i < 5; i++) {
            if (!checkCollision(currentX + kicks[i][0], currentY + kicks[i][1], currentType, newRotation)) {
                currentX += kicks[i][0];
                currentY += kicks[i][1];
                currentRotation = newRotation;
                playRotate();
                break;
            }
        }
    }
}

void hardDrop() {
    while (movePiece(0, 1)) {
        score += 2;
    }
}

void lockPiece() {
    for (int row = 0; row < 4; row++) {
        for (int col = 0; col < 4; col++) {
            if (getShape(currentType, currentRotation, row, col)) {
                int y = currentY + row;
                int x = currentX + col;
                if (y >= 0 && y < ROWS && x >= 0 && x < COLS) {
                    grid[y][x] = currentType + 1;
                }
            }
        }
    }
}

```



```

void render() {
    // Clear all LEDs
    FastLED.clear();

    // Draw locked pieces
    for (int row = 0; row < ROWS; row++) {
        for (int col = 0; col < COLS; col++) {
            if (grid[row][col] != 0) {
                int ledIndex = getLEDIndex(row, col);
                if (ledIndex >= 0) {
                    leds[ledIndex] = colors[grid[row][col] - 1];
                }
            }
        }
    }

    // Draw ghost piece (landing preview)
    int ghostY = currentY;
    while (!checkCollision(currentX, ghostY + 1, currentType, currentRotation)) {
        ghostY++;
    }

    for (int row = 0; row < 4; row++) {
        for (int col = 0; col < 4; col++) {
            if (getShape(currentType, currentRotation, row, col)) {
                int y = ghostY + row;
                int x = currentX + col;
                if (y >= 0 && y < ROWS && x >= 0 && x < COLS && grid[y][x] == 0) {
                    int ledIndex = getLEDIndex(y, x);
                    if (ledIndex >= 0) {
                        leds[ledIndex] = colors[currentType];
                        leds[ledIndex].fadeToBlackBy(200); // Dim ghost
                    }
                }
            }
        }
    }
}

```

```

void clearLines() {
    int linesCleared = 0;

    for (int row = ROWS - 1; row >= 0; row--) {
        bool fullRow = true;
        for (int col = 0; col < COLS; col++) {
            if (grid[row][col] == 0) {
                fullRow = false;
                break;
            }
        }

        if (fullRow) {
            linesCleared++;

            // Shift rows down
            for (int r = row; r > 0; r--) {
                for (int c = 0; c < COLS; c++) {
                    grid[r][c] = grid[r-1][c];
                }
            }

            // Clear top row
            for (int c = 0; c < COLS; c++) {
                grid[0][c] = 0;
            }

            row++; // Check this row again
        }
    }

    if (linesCleared > 0) {
        lines += linesCleared;
        int points[] = {0, 100, 300, 500, 800};
        score += points[linesCleared] * level;

        level = lines / 10 + 1;
        moveDelay = max(100, 1000 - (level - 1) * 100);

        playLineClear();
        updateLCD();
    }
}

```

```

// ===== MAIN LOOP =====
void loop() {
    // Test LED blink
    if (millis() - lastBlinkTime > 500) {
        blinkState = !blinkState;
        digitalWrite(TEST_LED_PIN, blinkState);
        lastBlinkTime = millis();
    }

    // Check START button
    if (!gameStarted && digitalRead(BTN_START) == LOW) {
        restartGame();
        delay(200);
        return;
    }

    // Game over state
    if (gameOver) {
        showGameOver();
        playGameOver();

        // Flash red
        static unsigned long lastFlash = 0;
        static bool flashState = false;

        if (millis() - lastFlash > 500) {
            flashState = !flashState;
            lastFlash = millis();

            if (flashState) {
                fill_solid(leds, NUM_LEDS, CRGB::Red);
            } else {
                FastLED.clear();
            }
            FastLED.show();
        }

        // Wait for START to restart
        if (digitalRead(BTN_START) == LOW) {
            delay(200);
            restartGame();
        }
        return;
    }
}

```

```

// Not started yet
if (!gameStarted) {
    playTetrisTheme();
    return;
}

// Paused
if (gamePaused) {
    handleButtons(); // Only check pause button
    return;
}

// Normal gameplay
handleJoystick();
handleButtons();

// Auto-drop piece
if (millis() - lastMoveTime > moveDelay) {
    if (!movePiece(0, 1)) {
        lockPiece();
        clearLines();
        spawnPiece();
        if (checkCollision(currentX, currentY, currentType, currentRotation)) {
            gameOver = true;
        }
    }
    lastMoveTime = millis();
}

render();
}

```

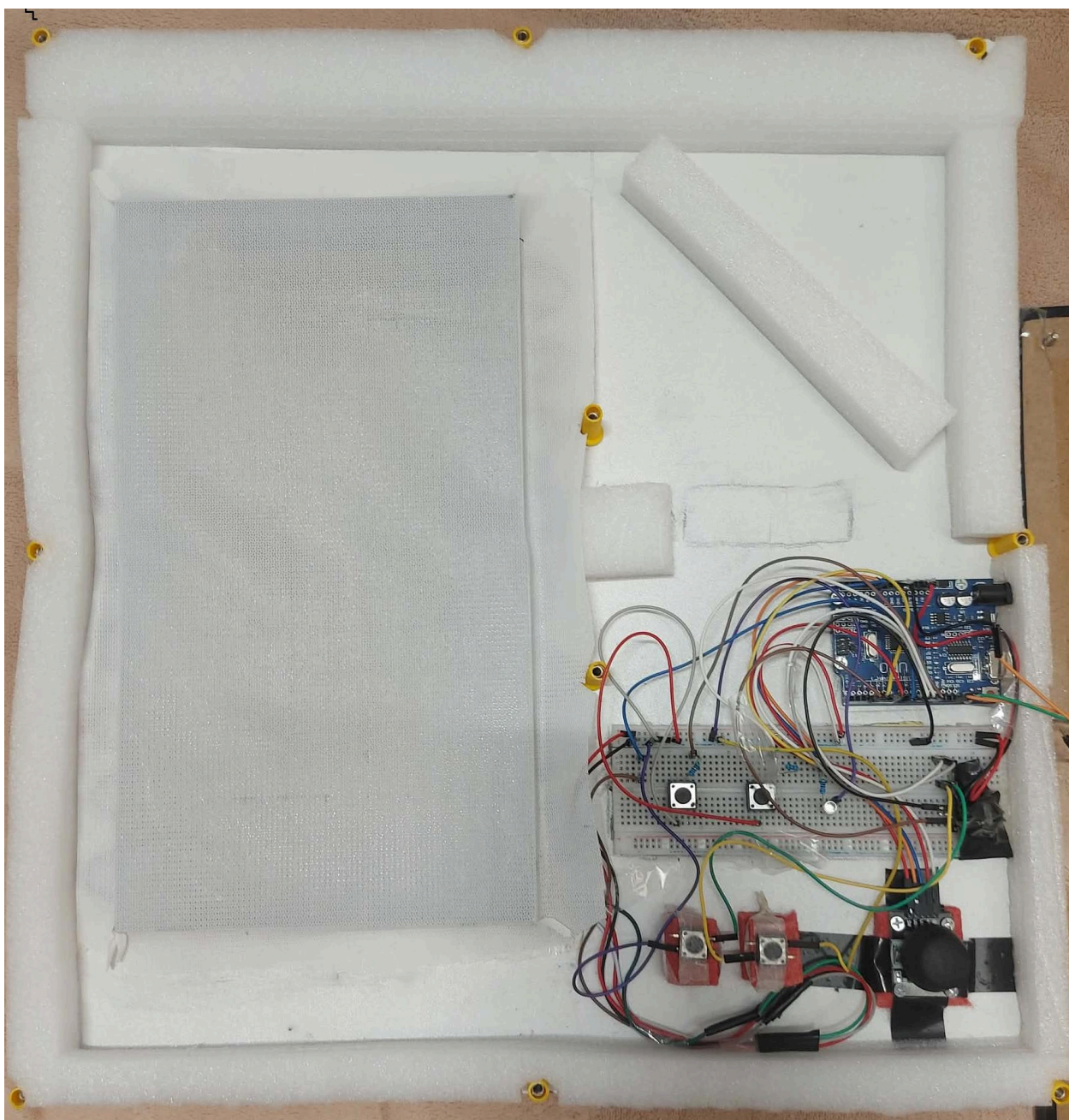
ТӨСӨЛ БҮТЭЭХ ҮЙЛ ЯВЦ

Схем /Ардуино, Эд ангиуд, нийт холбол



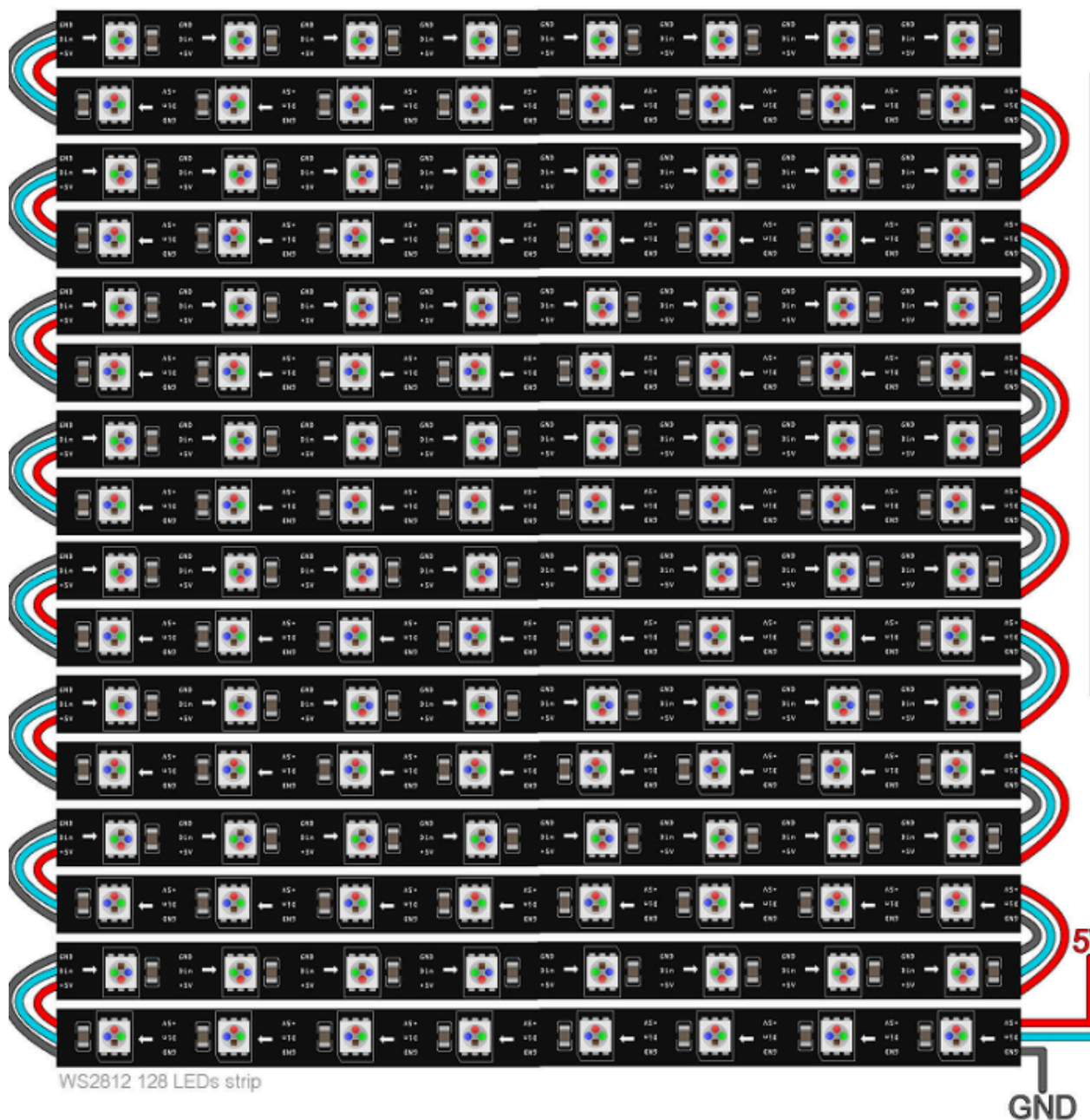
1.0

Манай төслийн гаднах үзэмж



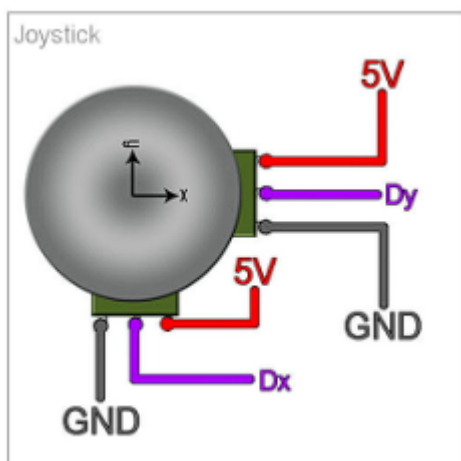
1.1

Манай төслийн доторх үзэмж



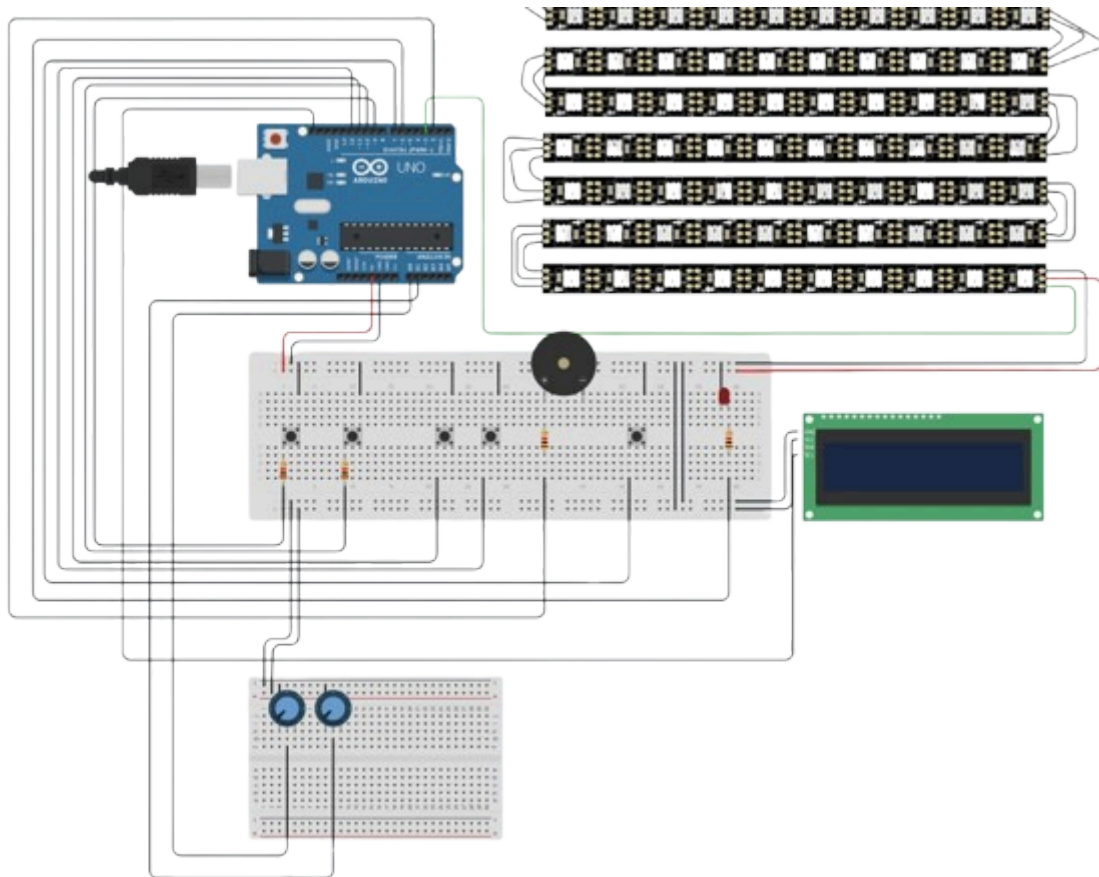
1.2

Энэ зурагнаас бидний **LED** гэрэл хэрхэн холбогдож байгааг харж болно.



1.3

Энэ зураг нь бидний joystick хэрхэн холбогдож байгааг үзүүлж байна.



1.4

Энэ бол манай төслийн холболт юм.

ТӨСЛИЙН ТЕСТ

Хүчдэл power

Манай төсөл нь Ардуино платформ дээр суурилсан тул 5 вольт хүчдэлээр ажилладаг.

Ажиглалт, Орчноос хамаарсан өөрчлөлт

Бидний ажигласнаар энэ төсөл нь цахилгаан шахах хүч муу байх үед бүх гэрэл хүрэлцэхгүй харахад хүндрэлтэй болгож байсан.

АШИГЛАСАН МАТЕРИАЛ

- Модон хавтан, шүрүүп, боолт
- Ардуино уно
- Холбогч утас

- I2C дэлгэц /LCD/
- Пиезо
- LED гэрэл
- Товчлууур, Joystick
- Талхны хавтан
- 3D Print хийлгэсэн хүснэгтэн бие
- Агааржуулагчийн тор

ТӨСЛИЙН ҮР ДҮН

Дүгнэлт

Бид энэхүү төслийг хэрэгжүүлснээр багийн уялдаа холбоо, хамтын ажиллагаагаа илүү бэхжүүлж, үүрэг хариуцлагаа ухамсарлан хүлээх чадвараа дээшлүүлж, авхаалж самбаагаа хамтдаа мэдэгдэхүйц сайжруулсан билээ.

[illegible]